



هوش مصنوعی

بهار ۱۴۰۵

استاد درس: دکتر احسان تن قطاری

مهلت ارسال: ۵ مرداد

پروژه درس هوش مصنوعی

- جزئیات پروژه‌ی نهایی درس در این سند قابل مشاهده است. نمره‌ی کل پروژه از ۴ نمره می‌باشد که در قالب گروه‌های ۲ نفری ارائه می‌شود.
- ابتدا یک فرم برای مشخص کردن گروه‌ها در کوئرا در اختیاران قرار داده می‌شود. مهلت پر کردن این فرم تا ساعت ۱۲ ظهر روز جمعه ۵ تیر می‌باشد. از هر گروه تنها یک نفر باید فرم را پر کند و همچنین دانشجویانی که فرم را پر نکرده باشند، به صورت تصادفی با یکدیگر همگروهی خواهند شد.
- بعد از مشخص شدن گروه‌ها، فرم انتخاب پروژه در ساعت ۸ شب روز جمعه ۵ تیر در کوئرا قرار داده خواهد شد و مهلت پر کردن آن تا ۱۲ ظهر فردای آن روز می‌باشد. گروه‌ها بر اساس زمان پر کردن این فرم برای تخصیص پروژه‌ها اولویت‌بندی می‌شوند. ظرفیت هر پروژه ۶ تیم می‌باشد.
- پروژه‌های ۱ و ۳ شامل ۱۰ درصد نمره‌ی امتیازی هستند که روی کل درس سرریز نمی‌شوند. پروژه‌های ۲ و ۴ شامل بخش امتیازی رقابتی هستند که برای تیم اول ۰.۴ نمره، تیم دوم ۰.۲ نمره و تیم سوم ۰.۱ نمره امتیازی روی کل درس اعمال می‌شود.
- ددلاین آپلود خروجی نهایی و گزارش پروژه تا ساعت ۲۳:۵۹:۵۹ روز دوشنبه ۵ مرداد می‌باشد. پس از آن اطلاعیه‌ای برای تحویل مجازی پروژه‌ها اعلام می‌شود.
- سوالات خود را راجع به روند پروژه در پیام مربوطه داخل کوئرا مطرح کنید.

صورت پروژه‌ها

پروژه ۱. شبکه‌های عصبی بازگشتی برای تشخیص فعالیت انسانی (۱۱۰ نمره)

۱. مقدمه و هدف

تصور کنید فقط از روی تکان‌های گوشی داخل جیب بتوان فهمید صاحبش در حال راه رفتن است، دویدن یا نشسته؛ در این پروژه دقیقاً همین سیستم را می‌سازید. هر گوشی یک شتاب‌سنج دارد که چندین بار در ثانیه شتاب بدن را در سه محور x ، y و z ثبت می‌کند، و این مقادیر پشت سر هم یک دنباله‌ی زمانی می‌سازند که الگوی حرکتی فرد در آن نهفته است. راه رفتن یک موج منظم، دویدن همان موج با دامنه‌ی بیشتر، و نشستن تقریباً یک خط صاف. مسئله‌ی تشخیص فعالیت انسانی (Human Activity Recognition) همین است که از روی این دنباله حدس بزنیم فرد چه می‌کند. چون ترتیب زمانی داده‌ها معنا می‌سازد و مدل باید نوعی حافظه داشته باشد، این مسئله تمرینی ایده‌آل برای شبکه‌های بازگشتی است.

شبکه‌های بازگشتی در درس پوشش داده نشده‌اند، پس پیش از شروع مرجع [۱] را مطالعه کنید که مقدمه‌ای روشن بر این شبکه‌هاست و پایه‌ی نظری لازم برای کل پروژه را فراهم می‌کند. هدف آن است که نخست یک شبکه‌ی بازگشتی ساده را از پایه و فقط با NumPy بسازید تا سازوکارش را واقعاً بفهمید، و سپس با معماری‌های آماده‌ی PyTorch مدل قوی‌تری برای همین مسئله بسازید و مقایسه کنید. در بخش اول استفاده از ماژول‌هایی مانند nn.RNN مجاز نیست؛ ساختن مدل از صفر فهم شما را از آنچه پشت ابزارهای آماده می‌گذرد بسیار عمیق‌تر می‌کند.

۱. مجموعه داده و آماده‌سازی

از مجموعه داده‌ی متن‌باز WISDM [۳] استفاده می‌کنید که ثبت شتاب‌سنج گوشی از چند کاربر هنگام شش فعالیت است: راه رفتن، دویدن، بالا و پایین رفتن از پله، نشستن و ایستادن. هر نمونه شامل شناسه‌ی کاربر، برجسب فعالیت و سه مؤلفه‌ی شتاب (a_x, a_y, a_z) است.

داده‌ی خام را نخست پنجره‌بندی کنید، یعنی دنباله‌ی هر کاربر را به قطعه‌های هم‌پوشان با طول ثابت T ببرید و برجسب هر پنجره را فعالیت غالب آن بگیرید؛ سپس داده را استاندارد کنید و میانگین μ و انحراف معیار σ را فقط از داده‌ی آموزش حساب کنید تا چیزی از داده‌ی آزمون نشت نکند: $x_{\text{norm}} = (x - \mu) / \sigma$. تقسیم آموزش و آزمون باید بر اساس کاربر باشد، یعنی هیچ کاربری هم‌زمان در هر دو نباشد؛ در غیر این صورت پنجره‌های مشابه در هر دو می‌افتند و دقت به‌شکلی فریبنده بالا می‌رود. چون توزیع کلاس‌ها نامتوازن است، در کنار دقت باید macro-F1 و ماتریس درهم‌ریختگی (Confusion Matrix) را نیز گزارش کنید.

۲. بخش اول: شبکه‌ی بازگشتی ساده از صفر

یک شبکه‌ی بازگشتی ساده با یک لایه‌ی پنهان را فقط با NumPy بسازید که طبقه‌بندی را بر پایه‌ی حالت آخرین گام h_T انجام می‌دهد:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad \hat{y} = \text{softmax}(W_{hy}h_T + b_y).$$

سپس پس‌انتشار در زمان (Back-Propagation Through Time) را پیاده کنید؛ چون W_{hh} میان همه‌ی گام‌ها مشترک است، گرادیانش جمع سهم همه‌ی گام‌هاست:

$$\delta_t = (W_{hh}^\top \delta_{t+1}) \odot (1 - h_t^2), \quad \frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^T \delta_t h_{t-1}^\top.$$

برای لایه‌ی خروجی از softmax و خطای Cross-Entropy استفاده کنید (می‌توانید پیاده‌سازی آماده‌ی آن‌ها را به کار ببرید) و برای پایداری از بریدن گرادیان (Gradient Clipping) بهره ببرید. پیش از آموزش، درستی پیاده‌سازی را با بررسی عددی گرادیان تأیید کنید: گرادیان تحلیلی را با تقریب تفاضل مرکزی $(\mathcal{L}(\theta + \varepsilon) - \mathcal{L}(\theta - \varepsilon)) / 2\varepsilon$ مقایسه کنید و خطای نسبی کمتر از 10^{-5} بگیرید. در گزارش هم منحنی کاهش خطای آموزش و هم نتیجه‌ی بررسی گرادیان را بیاورید.

۳. بخش دوم: معماری‌های آماده با PyTorch

ابتدا به‌عنوان بررسی سلامت نشان دهید شبکه‌ی NumPy شما و nn.RNN با وزن‌های یکسان خروجی تقریباً برابر می‌دهند. سپس دو مدل را آموزش و مقایسه کنید: همان شبکه‌ی ساده و LSTM. شبکه‌ی ساده روی دنباله‌های بلند به‌خاطر محو شدن گرادیان ضعیف است؛ LSTM [۲] این مشکل را با یک حالت سلولی و چند دروازه حل می‌کند:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t).$$

در گزارش توضیح دهید چرا این ساختار از محو شدن گرادیان جلوگیری می‌کند. در پایان یک LSTM دوسویه (Bidirectional) هم آموزش دهید و ببینید آیا دیدن ادامه‌ی پنجره به نتیجه کمک می‌کند.

۴. بخش سوم: تحلیل

نرم گرادیان را برحسب فاصله‌ی زمانی برای شبکه‌ی ساده و LSTM رسم کنید و تفاوتشان را در حفظ وابستگی‌های دور توضیح دهید. سپس با دو آزمایش کوتاه اثر طول پنجره T و اندازه‌ی لایه‌ی پنهان را بر دقت بسنجید و نتیجه را تفسیر کنید؛ برای مثال توضیح دهید چرا پنجره‌ی بسیار کوتاه یا بسیار بلند می‌تواند دقت را کم کند. همچنین F1 هر کلاس را جداگانه گزارش کنید تا مشخص شود مدل روی کدام فعالیت‌ها ضعیف‌تر است.

۵. تحویلی‌ها

خروجی را در پوشه‌ای به نام شماره‌ی دانشجویی خود فشرده (zip) کنید: یک نوت‌بوک ژوپیتِر (ipynb). شامل بخش اول و دوم با خروجی‌های اجراشده، و یک گزارش کوتاه PDF که روش کار، جدول‌ها، نمودارها و پاسخ پرسش‌های تحلیلی را در بر بگیرد. ضمیمه کردن وزن‌های مدل اختیاری است، اما مطمئن شوید نوت‌بوک به‌ترتیب و بدون خطا اجرا می‌شود.

۶. معیارهای ارزیابی

نتایج همه‌ی مدل‌ها را روی مجموعه‌ی آزمون تقسیم‌شده بر اساس کاربر در جدول زیر گرد آورید:

مدل	دقت	macro-F1	زمان آموزش
شبکه‌ی ساده (از صفر)			
(PyTorch) LSTM			
Bidirectional LSTM			

ماتریس درهم‌ریختگی بهترین مدل را رسم کنید و بگویید کدام دو فعالیت بیشتر با هم اشتباه می‌شوند. در گزارش به دو پرسش پاسخ دهید: اختلاف دقت میان تقسیم تصادفی و تقسیم بر اساس کاربر چقدر است و کدام صادقانه‌تر است؟ و چرا LSTM از شبکه‌ی ساده بهتر عمل می‌کند و این به محو شدن گرادیان چه ربطی دارد؟ تقسیم‌بندی نمره پروژه نیز به شکل زیر می‌باشد:

بخش	درصد نمره
بخش اول: شبکه‌ی بازگشتی از صفر و بررسی عددی گرادیان	۳۵٪
بخش دوم: معماری‌های آماده با PyTorch و مقایسه	۳۰٪
بخش سوم: تحلیل	۱۵٪
گزارش و کیفیت ارائه	۲۰٪
بخش امتیازی (اختیاری)	تا ۱۰٪

۷. بخش امتیازی (تا ۱۰٪)

با انجام یکی از دو مورد زیر تا ۱۰ درصد نمره‌ی اضافه می‌گیرید. نخست افزودن و آموزش معماری GRU (نسخه‌ی ساده‌تر LSTM با دو دروازه) و مقایسه‌ی آن با LSTM از نظر دقت و سرعت آموزش. گزینه‌ی دوم ارزیابی به روش Leave-One-Subject-Out روی زیرمجموعه‌ای از کاربران است که برآورد صادقانه‌تری از تعمیم مدل می‌دهد.

اگر با مفاهیمی مانند پس‌انتشار در زمان (BPTT)، محو شدن گرادیان، نقش دروازه‌های LSTM، نشت داده در تقسیم مبتنی بر کاربر، تفاوت macro-F1 و micro-F1، یا softmax و Cross-Entropy آشنا نیستید، مطالعه‌ی کوتاه آن‌ها پیش از شروع به شما کمک می‌کند.

مراجع

- [۱] Z. C. Lipton, J. Berkowitz, and C. Elkan, "A Critical Review of Recurrent Neural Networks for Sequence Learning," arXiv:1506.00019, 2015.
- [۲] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, 1997.
- [۳] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, "Activity Recognition using Cell Phone Accelerometers," ACM SIGKDD Explorations Newsletter, vol. 12, no. 2, 2011.

پروژه ۲. طراحی، پیاده‌سازی و ارزیابی جامع حملات و دفاع‌های خصمانه روی شبکه‌های عصبی عمیق (۱۰۰ نمره)

۱. مقدمه و هدف پروژه

تصور کنید یک سیستم تشخیص تصویر با دقت بالا دارید که با اطمینان یک گربه را از یک اتومبیل تشخیص می‌دهد. حال تنها چند پیکسل نامحسوس را در تصویر تغییر می‌دهیم — تغییری که چشم انسان اصلاً متوجه آن نمی‌شود — و ناگهان همان مدل با اطمینان بالا، آن گربه را «هواپیما» می‌خواند. این پدیده‌ای است که در دنیای واقعی رخ می‌دهد و به آن «مثال خصمانه» (Adversarial Example) می‌گویند.

در این پروژه شما با این آسیب‌پذیری عمیق شبکه‌های عصبی روبه‌رو می‌شوید. هدف این است که هم یاد بگیرید چگونه یک مدل را فریب دهید (حملات) و هم چگونه در برابر این فریب‌ها مقاوم شوید (دفاع‌ها). تمام پیاده‌سازی‌ها باید مستقیماً با PyTorch انجام شوند؛ استفاده از کتابخانه‌های آماده مانند Foolbox یا Torchattacks و یا هر کتابخانه دیگری که حمله‌ها را به طور خودکار انجام دهد، مجاز نیست، زیرا هدف اصلی، درک دقیق ریاضیات پشت این الگوریتم‌ها است.

۱. راه‌اندازی: مجموعه داده و مدل پایه (۱۰ نمره)

- **مجموعه داده:** از CIFAR-10 استفاده کنید؛ مجموعه‌ای با ۶۰۰۰۰ تصویر رنگی 32×32 در ۱۰ کلاس (هواپیما، اتومبیل، پرند، ...). برای تمام ارزیابی‌ها فقط از بخش تست (Test Split) استفاده کنید.

- **فضای پیکسلی $[0, 1]$:** هنگام بارگذاری تصاویر، مقادیر پیکسل را به بازه $[0, 1]$ نگه‌دارید (تقسیم بر ۲۵۵). حملات باید دقیقاً در همین فضا اعمال شوند تا بودجه نویز ϵ معنای فیزیکی داشته باشد (توضیح کامل در پیوست، بخش ۲).

- **کپسوله‌سازی مدل با لایه Normalize:** مدل پیش‌آموزش دیده ResNet-20 انتظار دارد تصاویر نرمال‌شده (با میانگین و انحراف معیار CIFAR-10) دریافت کند. اما ما می‌خواهیم حملات را روی تصاویر خام $[0, 1]$ اعمال کنیم. راه‌حل این است که لایه‌ای به نام Normalize بسازید که درون خود مدل نرمال‌سازی را انجام دهد:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

سپس این لایه را به عنوان اولین لایه در ابتدای شبکه قرار دهید:

```
model = Sequential(Normalize(), ResNet20)
```

لایه جدید

بدین ترتیب مدل کپسوله‌شده تصاویر $[0, 1]$ می‌گیرد و نرمال‌سازی را داخلی انجام می‌دهد.

۲. بخش اول: پیاده‌سازی حملات خصمانه (هر یک ۱۰ نمره)

در این بخش باید ۴ حمله‌ی جعبه‌سفید (White-Box) پیاده‌سازی کنید. در تمام حملات بودجه اختلال L_∞ برابر است با $\epsilon = 8/255$ ، مگر آنجایی که فرمول متفاوتی ذکر شده باشد.

(آ) حمله FGSM (روش علامت گرادیان سریع):

ساده‌ترین حمله است: یک قدم در جهتی برو که خطای مدل بیشینه شود. اگر $L(\theta, x, y)$ تابع زیان باشد، تصویر خصمانه با این فرمول ساخته می‌شود:

$$x_{\text{adv}} = \text{clip}_{[0,1]}(x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y)))$$

شما باید تأثیر مقادیر مختلف $\epsilon \in \{2/255, 4/255, 8/255, 16/255\}$ را برافت دقت بررسی و نمودار آن را رسم کنید.

(ب) حمله PGD (گرادیان کاهشی تصویری):

نسخه تکراری و قوی تر FGSM است: به جای یک قدم بزرگ، چند قدم کوچک برمی داریم و بعد از هر قدم، مطمئن می شویم که هنوز در بازه مجاز هستیم. فرآیند در گام های t به این صورت است:

$$x' = x + \mathcal{U}(-\epsilon, \epsilon), \quad x^{t+1} = \Pi_{x+B_\epsilon}(x^t + \alpha \cdot \text{sign}(\nabla_{x^t} L(\theta, x^t, y)))$$

که در آن Π تصویر روی کره ϵ حول تصویر اصلی است تا اختلال از حد مجاز خارج نشود، و α اندازه گام است.

جزئیات پیاده سازی: ۲۰ گام تکرار، شروع تصادفی (Random Start) برای شکستن تقارن، و clip به $[0, 1]$ بعد از هر گام.

(ج) حمله DeepFool:

FGSM و PGD با بودجه ثابت ϵ کار می کنند، اما DeepFool می پرسد: «کمترین تغییری که مدل را فریب دهد چقدر است؟» ایده اصلی این است که در هر گام، مرز تصمیم بین کلاس فعلی و نزدیک ترین کلاس دیگر را خطی فرض کنیم و تصویر را به سمت آن مرز هل دهیم. DeepFool یک حمله L_2 است (بودجه ثابتی ندارد) و معمولاً با اختلال بسیار کمتری از FGSM موفق به فریب مدل می شود.

جزئیات پیاده سازی: حلقه را تا تغییر پیش بینی مدل یا رسیدن به ۵۰ تکرار ادامه دهید. در گزارش، میانگین $\|\delta\|_2$ روی نمونه های موفق را گزارش کنید.

(د) حمله C&W (نسخه L_2):

پیچیده ترین و قوی ترین حمله این پروژه است. به جای دنبال کردن گرادیان، مسئله را به صورت یک مسئله بهینه سازی فرمول بندی می کند: کمترین نویز ممکن را پیدا کن که مدل را گول بزند:

$$\min_{\delta} \|\delta\|_2^2 + c \cdot f(x + \delta)$$

تابع هدف f بر اساس لاجیت های مدل تعریف می شود:

$$f(x') = \max \left(Z(x')_y - \max_{i \neq y} Z(x')_i, -\kappa \right)$$

که در آن $Z(x')$ لاجیت های خروجی شبکه و κ پارامتر اطمینان حمله است. وقتی $f < 0$ ، حمله موفق شده است.

برای اینکه خروجی حتماً در $[0, 1]$ بماند، بهینه سازی را روی متغیر کمکی w انجام دهید، نه مستقیماً روی δ :

$$x + \delta = \frac{1}{\gamma} (\tanh(w) + 1)$$

با این تغییر متغیر، $x + \delta$ همیشه بین ۰ و ۱ است. بهینه سازی را با Adam و برای ۱۰۰ تکرار انجام دهید.

۳. بخش دوم: پیاده سازی مکانیزم های دفاعی (هر یک ۱۰ نمره)

پس از اینکه دیدید مدل پایه چقدر آسیب پذیر است، باید ۳ استراتژی دفاعی را پیاده سازی کنید. هر دفاع رویکرد متفاوتی دارد و جایگاه مقایسه ای آن ها با هم نتیجه جالبی خواهد داشت.

(آ) آموزش خصمانه (Adversarial Training):

ایده ساده اما مؤثر است: اگر می خواهیم مدل در برابر حملات مقاوم باشد، آن را با نمونه های خصمانه آموزش دهیم. تابع هزینه به صورت زیر تغییر می کند:

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[\max_{\delta \in B_\epsilon} L(\theta, x + \delta, y) \right]$$

یعنی در هر گام آموزشی، بدترین اختلال ممکن را پیدا کن و مدل را مجبور کن همین را هم درست طبقه‌بندی کند.

جزئیات پیاده‌سازی: در هر اپک، برای هر دسته، ابتدا با PGD (۱۰ گام) نمونه‌های خصمانه بساز، سپس وزن‌های شبکه را بر اساس زیان این نمونه‌ها به‌روزرسانی کن. مدل را حداقل ۵ الی ۱۰ اپک روی CIFAR-10 آموزش ده.

(ب) هموارسازی تصادفی (Randomized Smoothing):

این دفاع در زمان تست کار می‌کند و مدل را تغییر نمی‌دهد. ایده اصلی: به جای یک پیش‌بینی، N بار تصویر را با نویز گاوسی مختلف به مدل بده و رأی‌گیری کن:

$$g(x) = \arg \max_c \mathbb{P}(f(x + \varepsilon) = c), \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$$

این روش یک گارانتی ریاضی دارد: اگر نویز خصمانه کمتر از حد مشخصی باشد، پیش‌بینی تغییر نخواهد کرد.

جزئیات پیاده‌سازی: برای هر تصویر، $N = 100$ نمونه نویزی با $\sigma = 0.25$ تولید کنید. کلاسی که بیشترین رأی را گرفت انتخاب کنید. **نکته مهم:** مدل پایه باید از قبل با نویز گاوسی فاین‌تیون شده باشد تا بتواند تصاویر نویزدار را درست طبقه‌بندی کند.

(ج) پالایش انتشاری (Diffusion Purification):

این دفاع نویز خصمانه را «پاک‌سازی» می‌کند قبل از اینکه تصویر به مدل برسد. ایده از مدل‌های انتشاری (Diffusion Models) می‌آید که در تولید تصویر استفاده می‌شوند. فرآیند دو مرحله دارد:

- **مرحله اول — اضافه کردن نویز (Forward Process):** به تصویر خصمانه x_{adv} در گام زمانی کوچک t نویز گاوسی اضافه کنید:

$$x_t = \sqrt{\alpha_t} x_{adv} + \sqrt{1 - \alpha_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, I)$$

این کار ساختار منظم نویز خصمانه را از بین می‌برد (چون نویز خصمانه بسیار کوچک است و در نویز گاوسی غرق می‌شود).

- **مرحله دوم — حذف نویز (Reverse Process):** با یک مدل انتشاری پیش‌آموزش‌دیده (مثلاً DDPM از HuggingFace)، مسیر معکوس را از t تا ۰ طی کنید تا تصویر تمیز بازسازی شود. این تصویر پالایش‌شده سپس به ResNet-20 داده می‌شود.

جزئیات پیاده‌سازی: از مدل google/ddpm-cifar10-32 از HuggingFace استفاده کنید. مقدار $t = 50$ برای تعادل میان پاک‌سازی و حفظ محتوای تصویر پیشنهاد می‌شود. این دفاع از نظر محاسباتی سنگین است، پس ارزیابی آن را روی زیرمجموعه‌ای کوچک‌تر (مثلاً ۲۰۰ تصویر) انجام دهید.

۴. نیازمندی‌های خروجی و معیارهای ارزیابی (۲۰ نمره)

خروجی نهایی باید شامل یک گزارش تحلیلی به همراه موارد زیر باشد:

- **ماتریس ارزیابی متقاطع:** دقت مقاوم هر مدل در برابر هر حمله را در جدول زیر ثبت کنید:
- **نمودار حرارتی (Heatmap):** جدول بالا را با Seaborn یا Matplotlib به صورت یک Heatmap رنگی نمایش دهید تا مقایسه بصری آسان‌تر باشد.
- **تصویرسازی نمونه‌ها:** برای هر یک از ۴ حمله، برای حداقل ۲ نمونه که حمله موفق بوده، سه تصویر را کنار هم نشان دهید: (الف) تصویر اصلی، (ب) نویز بزرگ‌نمایی‌شده، (ج) تصویر خصمانه نهایی. برای هر تصویر، برچسب پیش‌بینی شده و میزان اطمینان مدل را نیز ذکر کنید.
- **تحلیل توازن‌ها (Trade-offs):** در متن گزارش به سه سؤال زیر پاسخ دهید:

جدول ۱: دقت مقاوم^۱ مدل‌ها (درصد) در برابر حملات مختلف

C&W (L_2)	DeepFool	PGD	FGSM	Clean	دفاع / حمله
					مدل پایه (بدون دفاع)
					Adversarial Training
					Randomized Smoothing
					Diffusion Purification

- (آ) پدیده Robust Overfitting در آموزش خصمانه چیست؟ چگونه می‌توان آن را تشخیص داد و از آن جلوگیری کرد؟
- (ب) اضافه کردن DiffPure چقدر زمان استنتاج را افزایش می‌دهد؟ آیا این کندی در ازای بهبود دفاعی که ایجاد می‌کند توجیه دارد؟
- (ج) چرا Randomized Smoothing دارای گارانتی ریاضی است اما در تصاویر باکیفیت (بدون حمله) دقت کمتری نسبت به مدل اصلی دارد؟

۵. بخش رقابتی (اختیاری)

دقت مقاوم^۲

علاوه بر الزامات اصلی پروژه، یک رقابت نیز بین تیم‌هایی که این پروژه را انتخاب کرده‌اند برگزار خواهد شد. معیار رتبه‌بندی در این رقابت، **دقت مقاوم (Robust Accuracy) مدل Adversarial Training در برابر حمله C&W (L_2)** است.

به بیان دیگر، برای هر تیم مقدار موجود در خانه متناظر با سطر Adversarial Training و ستون C&W (L_2) از جدول ارزیابی به عنوان امتیاز رقابتی آن تیم در نظر گرفته خواهد شد.

- سه تیم برتر از نظر دقت مقاوم مدل Adversarial Training در برابر حمله C&W (L_2)، مقداری نمره امتیازی دریافت خواهند کرد.

در صورت برابری نتایج، کیفیت تحلیل ارائه‌شده در گزارش و بازتولیدپذیری نتایج به عنوان معیارهای تکمیلی برای تعیین رتبه نهایی در نظر گرفته خواهند شد.

^۲Robust Accuracy: درصد نمونه‌هایی که مدل پس از اعمال یک حمله خصمانه همچنان آن‌ها را به درستی طبقه‌بندی می‌کند.

۱. حملات جعبه سفید (White-Box Attacks)

در دنیای امنیت سیستم‌های هوش مصنوعی، میزان دسترسی مهاجم به اطلاعات مدل، توان حمله را تعیین می‌کند. سه سطح دسترسی رایج وجود دارد:

- **جعبه سفید (White-Box):** مهاجم به همه چیز دسترسی دارد — معماری، وزن‌ها، و مهم‌تر از همه گرادینان‌ها. قوی‌ترین نوع حمله است.
- **جعبه سیاه (Black-Box):** مهاجم فقط می‌تواند ورودی بدهد و خروجی بگیرد. وزن‌ها و گرادینان‌ها مخفی هستند.
- **جعبه خاکستری (Gray-Box):** دسترسی محدود — مثلاً معماری شناخته است اما وزن‌ها نه.

تمام حملات بخش اول این پروژه از نوع **جعبه سفید** هستند: شما برای ساختن تصویر خصمانه، مستقیماً گرادینان $\nabla_x L$ را از مدل می‌گیرید و از آن استفاده می‌کنید.

۲. چرا حمله باید در فضای $[0, 1]$ اعمال شود؟

این یکی از مهم‌ترین نکات فنی پروژه است که اشتباه در آن، نتایج را کاملاً بی‌معنا می‌کند.

بودجه نویز ϵ مشخص می‌کند هر پیکسل حداکثر چقدر می‌تواند تغییر کند. برای مثال $\epsilon = 8/255$ یعنی هر پیکسل مجاز است حداکثر ۸ واحد از ۲۵۵ واحد ممکن تغییر کند — تغییری که چشم انسان اصلاً متوجه آن نمی‌شود.

حال فرض کنید قبل از حمله، تصویر را نرمال می‌کنید (مثلاً میانگین کم می‌کنید و بر انحراف معیار تقسیم می‌کنید). مقادیر پیکسلی از بازه $[0, 1]$ خارج شده و مثلاً بین $-2/5$ و $+2/5$ می‌روند. حالا اگر $\epsilon = 8/255$ را اعمال کنید، این اضافه کردن عدد 0.031 به مقادیری که بین $-2/5$ و $+2/5$ هستند، دیگر معادل ۸ واحد از ۲۵۵ نیست — رابطه‌اش با تغییر واقعی پیکسل کاملاً به هم ریخته است.

راه حل (که در صورت پروژه توضیح داده شد): حمله را روی تصاویر خام $[0, 1]$ اعمال کنید. نرمال‌سازی را درون یک لایه Normalize داخل مدل قرار دهید، نه بیرون از آن.

۳. پدیده بیش‌برازش مقاوم (Robust Overfitting)

در آموزش معمولی، بیش‌برازش یعنی مدل روی داده‌های آموزشی خوب کار می‌کند اما روی داده‌های تست ضعیف است. در آموزش خصمانه، یک پدیده متفاوت رخ می‌دهد که «بیش‌برازش مقاوم» نام دارد.

چه اتفاقی می‌افتد؟ در اوایل آموزش خصمانه، دقت مدل در برابر حملات روی هر دو مجموعه آموزشی و تست بهتر می‌شود. اما پس از چند اپک، مدل شروع می‌کند به حفظ کردن نمونه‌های خصمانه آموزشی به شکلی که دقت روی نمونه‌های خصمانه آموزشی همچنان بالا می‌رود، اما دقت روی نمونه‌های خصمانه تست کاهش پیدا می‌کند.

چرا اتفاق می‌افتد؟ نمونه‌های خصمانه تولید شده توسط PGD در طول آموزش، به یک نوع «دیتاست مصنوعی» تبدیل می‌شوند که مدل آن‌ها را حفظ می‌کند. اما این نمونه‌ها کاملاً نماینده فضای کلی تهدیدها نیستند.

چگونه جلوگیری کنیم؟ در طول آموزش، دقت روی نمونه‌های خصمانه تست را نظارت کنید. هنگامی که این مقدار شروع به کاهش کرد، آموزش را متوقف کنید (Early Stopping). بهترین مدل برای ذخیره کردن، مدلی است که بالاترین دقت مقاوم روی مجموعه تست را دارد، نه آموزشی.

پروژه ۳. طراحی و پیاده‌سازی سیستم طبقه‌بند تصاویر زبان اشاره و مترجم بی‌درنگ به گفتار (۱۱۰ نمره)

۱. مقدمه و هدف پروژه

در این پروژه، شما یک سیستم کامل هوش مصنوعی را از مرحله پردازش داده‌های خام تا استقرار در یک محیط کاربردی (Real-time) توسعه خواهید داد. هدف اصلی، طراحی یک شبکه عصبی پیچشی (CNN) برای تشخیص حروف/کلمات از روی تصاویر دست در زبان اشاره، و سپس اتصال این مدل به یک سیستم بینایی ماشین برای ترجمه پیوسته حرکات به صوت است.

این پروژه به سه بخش اصلی تقسیم می‌شود و شما باید تمام مراحل را با دقت مستندسازی کرده و کد تمیز و قابل اجرا تحویل دهید.

۲. فاز اول: تحلیل و پیش‌پردازش مجموعه داده (۲۵٪ نمره)

مجموعه داده‌ای حاوی تصاویر کلاس‌بندی شده از زبان اشاره در اختیار شما قرار خواهد گرفت. پیش از طراحی مدل، درک درست از داده‌ها الزامی است.

- **بارگذاری و اکتشاف داده (EDA):** اسکرپتی بنویسید که داده‌ها را بارگذاری کرده و توزیع کلاس‌ها را بررسی کند. آیا داده‌ها متوازن (Balanced) هستند؟ برای هر کلاس، تعداد نمونه‌ها را در قالب یک نمودار میله‌ای (Bar Plot) رسم کنید.
- **مصورسازی نمونه‌ها:** از هر کلاس، حداقل یک تصویر تصادفی را انتخاب کرده و در یک شبکه (Grid) با برجسب مربوطه نمایش دهید.
- **پیش‌پردازش (Preprocessing):** تصاویر باید برای ورود به شبکه عصبی استاندارد شوند. ابعاد تصاویر را یکسان‌سازی کرده (مثلاً تغییر اندازه به 64×64 یا 128×128) و مقادیر پیکسل‌ها را نرمال‌سازی کنید (نگه داشتن در بازه $[0, 1]$ یا استانداردسازی با میانگین و واریانس).
- **تقسیم‌بندی داده‌ها:** داده‌ها را به سه بخش آموزش (Train)، اعتبارسنجی (Validation) و تست (Test) با نسبت‌های استاندارد (مانند ۸۰-۱۰-۱۰) تقسیم کنید.

۳. فاز دوم: طراحی و آموزش شبکه عصبی پیچشی (۶۰٪ نمره + ۱۰٪ نمره امتیازی)

در این بخش باید یک معماری CNN مناسب برای استخراج ویژگی‌های مکانی از تصاویر دست طراحی کنید.

- **طراحی معماری شبکه:** یک مدل CNN شامل لایه‌های Convolution، MaxPooling و Fully Connected پیاده‌سازی کنید. دلیل انتخاب تعداد لایه‌ها، ابعاد فیلترها و توابع فعال‌ساز (مانند ReLU) را در گزارش خود شرح دهید.
- **تنظیم پارامترها (Hyperparameter Tuning):** مدل شما باید با پارامترهای مناسبی آموزش ببیند. در گزارش خود تأثیر Learning Rate، Batch Size و نوع بهینه‌ساز (Adam یا SGD) را بررسی کنید.
- **ارزیابی مدل:** نمودارهای Loss و Accuracy را برای داده‌های آموزش و اعتبارسنجی در طول Epochها رسم کنید. پس از پایان آموزش، مدل را روی داده‌های تست ارزیابی کرده و **ماتریس درهم‌ریختگی (Confusion Matrix)** را چاپ کنید تا مشخص شود مدل در تشخیص کدام حروف بیشتر اشتباه می‌کند.
- **بخش امتیازی (تا ۱۰٪):** دانشجویانی که از تکنیک‌های پیشرفته‌تر برای بهبود دقت و جلوگیری از Overfitting استفاده کنند، نمره مازاد دریافت خواهند کرد. مواردی مانند:
 - استفاده از اتصالات میان‌بر (Residual Connections).
 - پیاده‌سازی Batch Normalization و Dropout در معماری.
 - استفاده از تکنیک‌های افزونگی داده (Data Augmentation) مانند چرخش، تغییر روشنایی و انتقال تصویر.

۴. فاز سوم: سیستم ترجمه بلادرنگ به گفتار (۱۵٪ نمره)

مدل آموزش دیده شما نباید فقط در محیط تعاملی نوت‌بوک بماند. در این بخش باید یک ابزار کاربردی بسازید که از دوربین سیستم (وب‌کم) استفاده کند.

- **دریافت تصویر با OpenCV:** یک حلقه پردازشی بنویسید که فریم‌ها را از وب‌کم خوانده، کادر (Bounding Box) مشخصی برای قرار گرفتن دست کاربر رسم کند و تنها محتوای آن کادر را به مدل شما پاس دهد.

- **منطق تشخیص پیوسته:**

(آ) مدل در هر فریم پیش‌بینی خود را انجام می‌دهد. اگر یک حرف/کلمه برای ۲ ثانیه متوالی با اطمینان بالا تشخیص داده شد، آن کاراکتر به رشته متنی خروجی اضافه شود و روی صفحه نمایش داده شود.

(ب) اگر برای مدت ۵ ثانیه کادر خالی بود (تشخیص کلاس Background یا Nothing)، سیستم باید رشته متنی جمع‌آوری شده را نهایی کند.

- **تبدیل متن به گفتار (TTS):** پس از نهایی شدن رشته متنی در مرحله قبل، با استفاده از کتابخانه‌هایی مانند pyttsx3 یا gTTS، متن را به صورت صوتی پخش کنید و سپس رشته را برای کلمات بعدی خالی کنید.

۵. الزامات تحویل و داوری پروژه

رعایت ساختار تحویل پروژه برای اجرای اسکریپت‌های داوری خودکار الزامی است. عدم رعایت این موارد منجر به کسر نمره خواهد شد:

- **ساختار فایل‌ها:** فایل‌های خود را در یک فایل زیپ به نام AI-Project3-StudentID1-StudentID2 قرار دهید.

- **فایل‌های مخفی سیستم عامل:** پیش از فشرده‌سازی، اکیداً مطمئن شوید که پوشه‌های اضافی سیستم عامل مانند __MACOSX یا فایل‌های DS_Store در فایل نهایی وجود نداشته باشند.

- **محتویات الزامی:**

(آ) نوت‌بوک ژوپیتِر (ipynb). شامل فاز ۱ و ۲ با تمام خروجی‌های اجرا شده.

(ب) فایل اسکریپت پایتون (main.py) برای اجرای فاز ۳ (بی‌درنگ).

(ج) فایل وزن‌های ذخیره شده مدل.

(د) فایل مستندات (گزارش) به فرمت PDF شامل توضیحات معماری شبکه و اقدامات انجام شده برای

بهبود دقت و جلوگیری از بیش‌برازش. همچنین نمودارها و خروجی‌های به دست آمده در نوت‌بوک باید در این بخش تحلیل شوند. بخشی از نمره هر فاز مربوط به ارائه گزارش کار مناسب برای آن است.

۶. منابع و مقالات پیشنهادی

برای درک بهتر مفاهیم لایه‌های پیچشی، شبکه‌های عمیق‌تر و بهینه‌سازی مدل برای کاربردهای بی‌درنگ، مطالعه مقالات زیر توصیه می‌شود:

- **مقاله کلاسیک AlexNet (آشنایی با ساختار اولیه CNN ها):**

این مقاله نقطه عطف بینایی ماشین با یادگیری عمیق است و مفاهیمی مانند Dropout و تابع فعال‌ساز ReLU را محبوب کرد.

[\[لینک دسترسی\]](#)

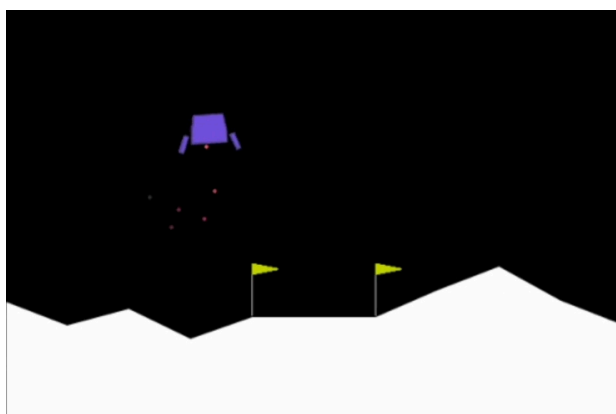
- **مقاله ResNet (بخش امتیازی - اتصالات میان‌بر):**
برای درک چگونگی عملکرد Residual Connections و حل مشکل محو شدن گرادیان (Vanishing Gradient) در شبکه‌های عمیق، این مقاله می‌تواند کمک‌کننده باشد.
[\[لینک دسترسی\]](#)
- **مقاله MobileNets (فاز سوم - مدل‌های بهینه برای سیستم‌های بلادرنگ):**
از آنجا که در فاز سوم باید مدل را به صورت زنده روی وب‌کم اجرا کنید، آشنایی با معماری‌های سبک و سریع مانند MobileNet که برای سیستم‌های لبه (Edge Devices) طراحی شده‌اند، بسیار مفید خواهد بود.
[\[لینک دسترسی\]](#)

۱. مقدمه و هدف پروژه

در این پروژه، شما وظیفه دارید یک عامل هوشمند (Agent) را با استفاده از الگوریتم Deep Q-Network (DQN) از پایه پیاده‌سازی و آموزش دهید تا بتواند یک سفینه فضایی را به سلامت روی سطح ماه فرود آورد. استفاده از هرگونه کتابخانه آماده یادگیری تقویتی (مانند StableBaselines، Ray RLlib و موارد مشابه) اکیداً ممنوع است و تمامی بخش‌های الگوریتم باید توسط خود شما طراحی و کدنویسی شود.

۲. پیش‌نیازها و محیط بازی

تمامی پیش‌نیازهای اولیه برای اجرای این پروژه، شامل کدهای پایه محیط بازی و فایل requirements.txt (حاوی لیست و نسخه دقیق کتابخانه‌های مجاز پایتون)، از طریق لینک زیر در اختیار شما قرار می‌گیرد. فایل game.py فیزیک محیط، دینامیک سفینه و سیستم امتیازدهی را مدیریت می‌کند. تصویر زیر نمایی از فضای بازی را نشان می‌دهد:



شکل ۱: فضای بازی فرودگر ماه

عامل شما باید با دریافت وضعیت فعلی سفینه (شامل مختصات، سرعت خطی، زاویه، سرعت زاویه‌ای و وضعیت پایه‌ها) تصمیم بگیرد که در هر لحظه کدام یک از ۴ موتور (موتور اصلی، موتور چپ، موتور راست یا خاموش بودن موتورها) را فعال کند تا سفینه بدون سقوط، با کمترین مصرف سوخت و به نرمی در پد مشخص شده فرود بیاید.

۳. مقالات مرجع (مطالعه الزامی)

برای درک دقیق الگوریتم و نحوه پیاده‌سازی مکانیزم‌هایی مانند Experience Replay و Target Network، مطالعه مقالات زیر الزامی است. معماری شبکه شما باید بر اساس مفاهیم این دو مقاله طراحی شود:

- مقاله اول (معرفی اولیه معماری و Experience Replay):

– عنوان: Playing Atari with Deep Reinforcement Learning

– نویسندگان: Volodymyr Mnih et al.

– لینک دانلود: <https://arxiv.org/abs/1312.5602>

- مقاله دوم (معرفی Target Network برای پایداری آموزش):

- عنوان: Human-level control through deep reinforcement learning

- نویسندگان: Volodymyr Mnih et al.

- لینک دانلود: <https://www.nature.com/articles/nature14236>

۴. نحوه اتصال کدهای شما به محیط بازی

ارتباط عامل شما با محیط تنها از طریق دو متد اصلی انجام می‌شود. نیازی به تغییر کدهای game.py ندارید؛ صرفاً باید کلاس آن را در کدهای خود ایمپورت کنید:

- متد reset(): در ابتدای هر اپیزود فراخوانی می‌شود تا بازی به حالت اولیه برگردد و state اولیه را به عامل بدهد.
- متد step(action): اکشن انتخابی شبکه شما را در محیط اعمال کرده و خروجی (next_state, reward, done) را برمی‌گرداند.

```
from game import LunarLanderEnv

env = LunarLanderEnv()
state = env.reset()

# Inside the main training loop
next_state, reward, done = env.step(action)
```

۵. ساختار فایل‌های تحویلی

پس از اتمام فرآیند آموزش، گروه‌ها موظف‌اند خروجی نهایی خود را در قالب یک فایل فشرده (ZIP) تحویل دهند. ساختار فایل‌های داخل این پوشه باید دقیقاً به شکل زیر باشد و عدم رعایت این ساختار موجب کسر نمره خواهد شد:

۱. model.py: فایلی که صرفاً شامل کلاس‌های مربوط به معماری شبکه عصبی (Policy Net و Target Net) است.

۲. agent.py: فایلی شامل کلاس اصلی Agent، پیاده‌سازی حافظه تجربه (Replay Buffer)، منطق انتخاب اکشن (Greedy- ϵ) و تابع Backpropagation.

۳. train.py: اسکریپت اصلی اجرای حلقه آموزش. این فایل باید با اجرای مستقیم، فرآیند یادگیری را آغاز کرده و در نهایت وزن‌ها را ذخیره کند.

۴. test.py: اسکریپتی برای ارزیابی مدل آموزش‌دیده. این فایل باید وزن‌های ذخیره‌شده را بارگذاری کرده و محیط بازی را به صورت گرافیکی رندر کند تا عملکرد سفینه قابل مشاهده باشد.

۵. weights.pth (یا h5): فایل حاوی وزن‌های نهایی شبکه عصبی آموزش‌دیده که توانسته بالاترین امتیاز را کسب کند.

۶. Report.pdf: گزارش نهایی پروژه شامل:

- توضیح معماری شبکه و توابع فعال‌ساز.
- نمودار روند آموزش (محور X: اپیزودها، محور Y: مجموع پاداش).
- نمودار میانگین متحرک (Moving Average) برای نشان دادن همگرایی.

- جدول کامل Hyperparameter های استفاده شده (نرخ یادگیری، ضریب γ ، ظرفیت بافر و نرخ کاهش ϵ).
- پاسخ به سوالات مفهومی مندرج در بخش ۸.

۶. رقابت و نمرات امتیازی

کدهای تحویلی تمامی تیم‌ها در یک محیط ایزوله، بدون رندر گرافیکی و با تنظیم یک Seed یکسان برای تمامی گروه‌ها ارزیابی خواهد شد. مدل هر تیم در چندین اپیزود متوالی تست شده و میانگین امتیازات محاسبه می‌گردد. بر این اساس، به سه تیمی که بالاترین عملکرد را داشته باشند، نمره امتیازی در درس تعلق می‌گیرد.

۷. بارم‌بندی و ارزیابی (مجموع: ۱۰۰ نمره + نمره امتیازی)

توجه بسیار مهم: تخصیص تمامی نمرات این پروژه مشروط به تحویل کامل فایل‌ها و اجرای بدون خطای کدها است. در صورتی که پروژه تحویل داده نشود یا کدهای تحویلی قابلیت اجرا نداشته باشند، نمره کل این پروژه صفر لحاظ خواهد شد.

الف) بخش پیاده‌سازی و کدنویسی (۶۰ نمره)

این بخش به بررسی صحت منطق الگوریتم، رعایت اصول برنامه‌نویسی و عملکرد مدل می‌پردازد:

- **منطق اصلی الگوریتم (۲۰ نمره):** پیاده‌سازی دقیق حافظه تجربه (Replay Buffer)، مکانیزم انتخاب اکشن (ϵ -Greedy) و به‌روزرسانی وزن‌ها (Backpropagation) در فایل `agent.py`.
- **معماری شبکه عصبی (۱۰ نمره):** پیاده‌سازی صحیح کلاس‌های شبکه اصلی و شبکه هدف (Target Network) در فایل `model.py`.
- **حلقه آموزش (۱۰ نمره):** تعامل درست عامل با محیط، مدیریت اپیزودها، و پیاده‌سازی مکانیزم انتقال وزن‌ها به شبکه هدف در فایل `train.py`.
- **ارزیابی و عملکرد مدل (۱۰ نمره):** اجرای موفق فایل `test.py` با بارگذاری فایل `weights.pth`؛ مدل باید همگرایی اولیه را نشان دهد و عملکردی بهتر از یک عامل کاملاً تصادفی (Random Agent) داشته باشد.
- **کیفیت کد و رعایت ساختار (۱۰ نمره):** کامنت‌گذاری مناسب، خوانایی کدها، نام‌گذاری اصولی متغیرها و رعایت دقیق ساختار فایل‌های خواسته‌شده (عدم وجود کدهای اضافی یا اسپاگتی).

ب) بخش مستندات و گزارش کار (۴۰ نمره)

داکیومننت شما نشان‌دهنده میزان درک شما از کاری است که انجام داده‌اید. فایل `Report.pdf` باید شامل موارد زیر باشد:

- **نمودارها و تحلیل روند آموزش (۲۰ نمره):** رسم دقیق و خوانای نمودارهای مجموع پاداش و میانگین متحرک (Moving Average). علاوه بر رسم نمودار، باید نوسانات و روند همگرایی یا واگرایی مدل را در گزارش تحلیل کنید (مثلاً چرا در اپیزودهای خاصی افت پاداش داشته‌اید).
- **شرح معماری (۱۰ نمره):** توضیح شفاف لایه‌های شبکه، ابعاد ورودی/خروجی و توابع فعال‌ساز استفاده شده به همراه دلیل انتخاب آن‌ها.
- **هایپرپارامترها، تنظیمات و پاسخ به سوالات مفهومی (۱۰ نمره):** ارائه جدول کامل پارامترها و همچنین تحلیل و پاسخ‌دهی دقیق به ۴ سوال مفهومی بخش ۸.

ج) رقابت و نمره امتیازی (Bonus)

همان‌طور که در بخش ۶ ذکر شد، کدهای تمامی تیم‌ها در یک محیط ایزوله تست خواهند شد. به سه تیمی که بالاترین میانگین امتیاز (پاداش) را در اپیزودهای ارزیابی س کنند، ۱۰ نمره امتیازی (مازاد بر سقف ۱۰۰ نمره) در نمره نهایی پروژه تعلق خواهد گرفت.

۸. سوالات مفهومی (پاسخ در گزارش کار الزامی است)

برای اطمینان از درک عمیق مفاهیم الگوریتم DQN، پاسخ تحلیلی به سوالات زیر در فایل گزارش کار (Report.pdf) الزامی است:

۱. **چالش همبستگی داده‌ها (Experience Replay):** چرا در الگوریتم DQN از مکانیزم حافظه تجربه استفاده می‌کنیم؟ اگر شبکه عصبی را مستقیماً با تجربیات متوالی و در لحظه (بدون استفاده از بافر) آموزش دهیم، چه مشکلی در فرآیند همگرایی شبکه پیش می‌آید؟
۲. **پایداری در آموزش (Target Network):** دلیل استفاده از دو شبکه مجزا (شبکه اصلی و شبکه هدف) چیست؟ مفهوم «هدف متحرک» (Moving Target) در یادگیری تقویتی عمیق چیست و به‌روزرسانی با تاخیر شبکه هدف چگونه این مشکل را حل می‌کند؟
۳. **توازن بین جستجو و بهره‌برداری (Exploration vs. Exploitation):** استراتژی کاهش ϵ (اپسیلون) چه نقشی در یادگیری عامل دارد؟ تحلیل کنید که اگر نرخ کاهش این پارامتر (Decay Rate) بیش از حد سریع یا بیش از حد کند باشد، رفتار سفینه فضایی در طول آموزش چه تغییری می‌کند.
۴. **تحلیل مقادیر (Q-Values):** خروجی شبکه عصبی شما در لایه آخر دقیقاً نمایانگر چه مقادیری است؟ اگر سفینه در ارتفاع بالا و با سرعت زیاد در حال سقوط آزاد باشد، منطقاً انتظار دارید مقدار خروجی شبکه برای اکشن «روشن کردن موتور اصلی» در مقایسه با سایر اکشن‌ها چگونه باشد و چرا؟